

CS2109S: Introduction to AI and Machine Learning

Lecture 1:
Intro to CS2109S and AI

13 January 2026

DO NOT CLOSE YOUR POLLEVERYWHERE APP

There will be activities ahead

Rights Infringements on NUS Course Materials

All course participants (including permitted guest students) who have access to the course materials on Canvas or any approved platforms by NUS for delivery of NUS modules are not allowed to re-distribute the contents in any forms to third parties without the explicit consent from the module instructors or authorized NUS officials

Instructors



Muhammad Rizki Maulana

rizki@nus.edu.sg

Deep Reinforcement Learning



Conghui Hu

conghui@nus.edu.sg

Deep Learning, Computer Vision



Patrick Rebentrost

cqtfpr@nus.edu.sg

Quantum Computing, Quantum
Machine Learning

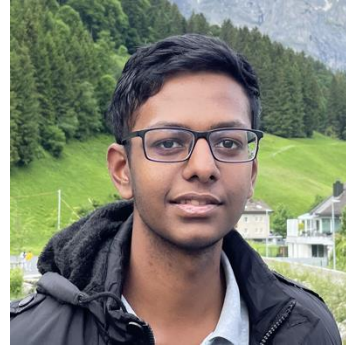
Teaching Assistant



Danton



Ian



Kannan



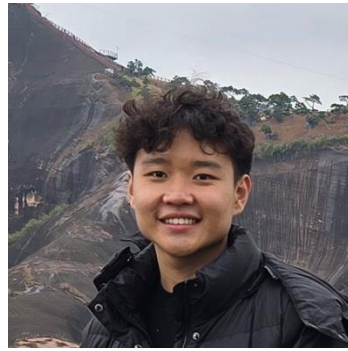
Lu Dongyue



Ee Chern



Chuen Yang



Wai Hong



Kum Chai Yin



Vidushi Jaidka



Minh Nguyen

Teaching Assistant



Nhat Chung



Zhang Jiaxiang



Benson



Soujanya
Lohanathen



Haoxiang



Shen Jingtao



Fengrui



Terry Si



Cheka



Chen Hanlin

Teaching Assistant



Loh Kwong Weng



Zwe Zeya



Tanya Warriier



Gan Pang Yen



Lam Jiu Fong



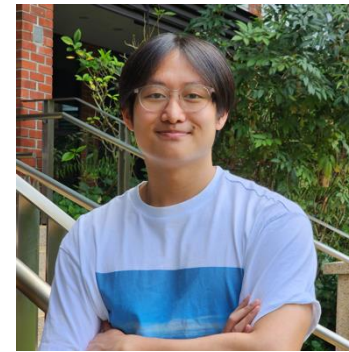
Teng Yee Shin



Siyan



Kieron



Bryan



Suhail Loya

Teaching Assistant



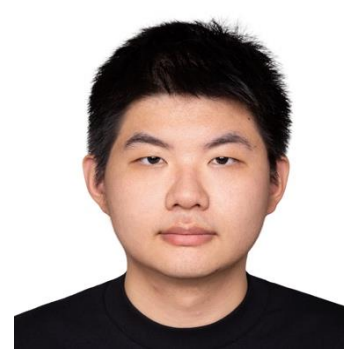
Ryan Reno Lim



Phi



Aprup Kale



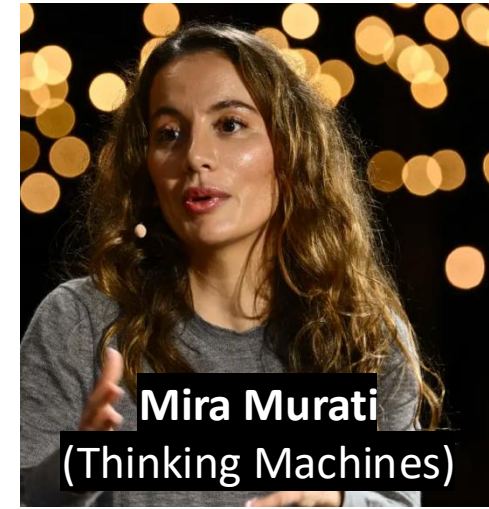
Song Chenan



Wang Junwu

Why do we want to learn about AI & ML?

AI is Changing the World



Videos:

- <https://www.youtube.com/watch?v=MAcEAAjgY3U> (A bit exaggerated)
- <https://www.youtube.com/watch?v=byYIC2cagLw>



Why? (1)

- For all: Learn some of the most important foundations of AI & ML
- Higher-level courses are built upon the knowledge you'll learn here
 - CS3263 Foundations of Artificial Intelligence
 - CS3264 Foundations of Machine Learning
 - CS4246/CS5446 AI Planning and Decision Making
 - CS5339 Theory and Algorithms for Machine Learning
 - ... even 6000 (PhD level) courses
- Aspiring researchers: These topics are important for state-of-the-art research as well. For both AI and STEM researchers.

Why? (2)

- Companies are incorporating AI & ML technologies in their products
- Software developers are expected to know about the fundamentals.
- As CS graduate you may be asked about:
 - **What** AI & ML tech are available
 - **How**
 - ... to use them
 - ... to interpret their outputs
 - **When** they are appropriate / not appropriate
 - **Why** they work / don't work

Intro to CS2109S

CS2109S

An introductory course designed to cover the fundamental topics in AI and ML.

What to Expect:

- A wide range of **introductory** topics in "classical" AI and ML.
- **Practical knowledge** on implementing basic AI and ML algorithms and applying them to **solve problems**.
- **Conceptual understanding** of key principles and foundational concepts in AI and ML.
- Mathematical formalization and discussion of the topics.

What Not to Expect:

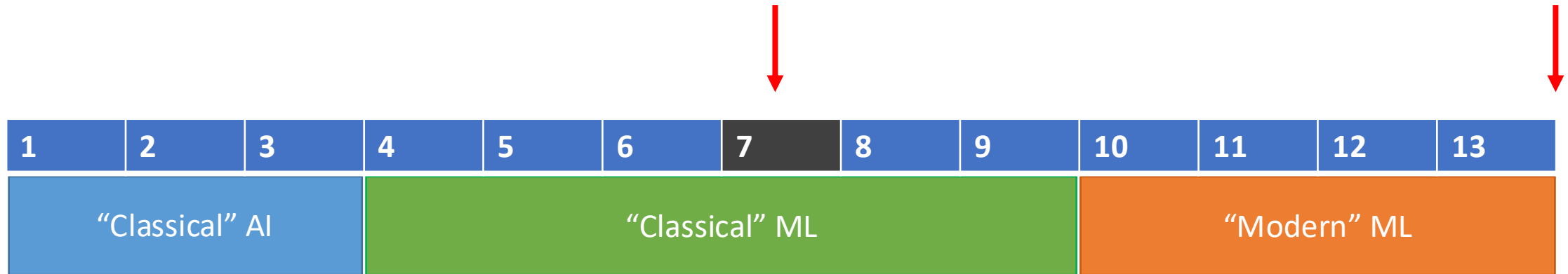
- A course focused on theoretical depth with rigorous proofs, mathematics, and theorems.
- Extensive coverage of the latest advancements in AI and ML.

These are features, not bugs.

Overview

Midterm
(Monday, 2 March 2026)

Final Exam
(Saturday, 2 May 2026)



Search Algorithms

- Uninformed search: BFS, DFS
- Informed search: A*
- Local Search: Hill Climbing
- Adversarial search: Minimax

"Classical" ML

- Decision Trees
- Linear/Logistic Regression
- Regularization and Kernels
- Neural networks

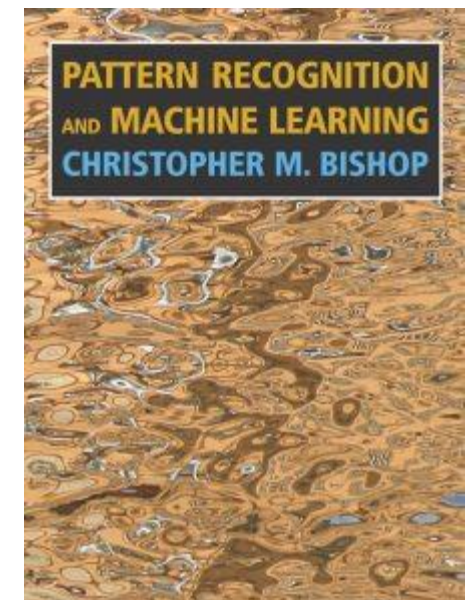
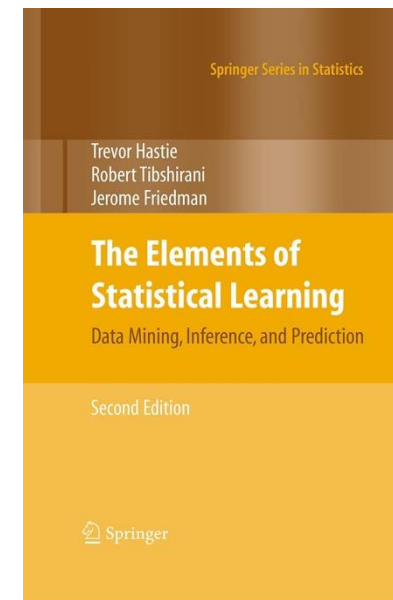
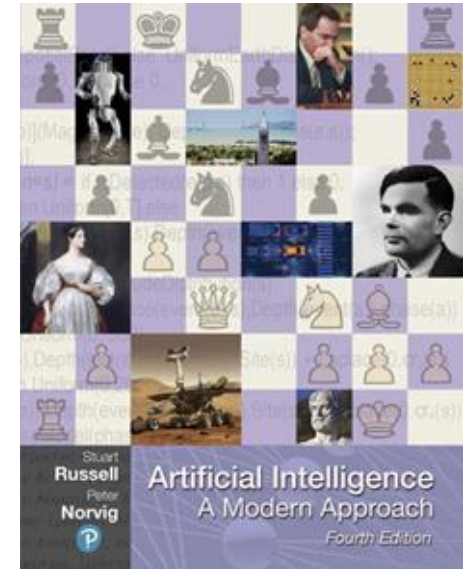
"Modern" ML

- Deep Learning
- Sequential Data
- Intro to unsupervised (deep) learning

Textbooks

- Russell and Norvig (2021)
Artificial Intelligence: A Modern Approach (4th Ed)
- Jerome H. Friedman, Robert Tibshirani, and Trevor Hastie (2009)
The Elements of Statistical Learning (2nd Ed)
- Christopher Bishop (2006)
Pattern Recognition and Machine Learning

There is no need to buy these books.



Learning Management System



Only for **Gradebook** to record marks
for checking and **webcast**

Lecture Slides

Lecture slides will be released **after** the lecture.

Why?

- We are preparing the slides before the lecture
- Swiping through the slides to catch up or preview ahead can distract you from following the lecture

NUS Policy

Office of Student Conduct's Circular No.1 of 2026 - Student Conduct and Academic Integrity

“The University takes a strong stance against plagiarism and considers it a serious form of academic dishonesty. Any student found to have taken the work or ideas of another person or entity, including through the use of generative Artificial Intelligence (AI), and passed them off as one’s own without proper attribution will be subject to disciplinary action. Please refer to the updated NUS Plagiarism Policy.”

NUS Plagiarism Policy:

- <https://myportal.nus.edu.sg/studentportal/student-discipline/all/docs/NUS-Plagiarism-Policy.pdf>

Course policy:

- <https://canvas.nus.edu.sg/courses/85417/pages/course-policies>

Collaboration Policy

- Collaborate on assessments **only** to the extent of **discussing ideas**, not (potential) implementation or (potential) code. All discussions must be **cited**.
- It is okay to incorporate a few lines of code that you find online or elsewhere into your own code, **provided that those lines are not themselves solutions** to assigned work and that you **cite** the lines' origins.
- Avoid sharing, consulting, discussing, or using (parts of) solution codes or their essence **through any means**, including chat, spoken words, whiteboard, video, or brief glances. This sharing policy also applies after you have completed the module.
- Do not view or use another person's solution as the basis for your own solution.

If you have any doubts about whether your actions are permissible, please contact us.

AI Policy

- Using AI is permitted as long as proper citations are provided.
 - If it is possible to share a link to the conversation (e.g., ChatGPT, Bing), then provide the link; if not (e.g., GitHub Copilot), then take screenshots
 - Document any changes made to the code generated by AI
- It is advisable to use AI-generated code as it is; alteration/obfuscation is strongly discouraged.

Citation Policy

The motivation for this citation policy is that you clearly identify the origins of each part of your submission.

- All **discussions, few lines of code from online sources, and AI conversations** must be cited **at the point in the code** (e.g., beginning of a code block) where they have influenced your code **via a python comment**.
- For **conceptual (non-code) questions**, provide the appropriate citations in the respective answer field. You may use the # character to indicate a comment.
- Provide any screenshots in the "Reference" section.

On Plagiarism

- Any plagiarism or academic dishonesty will be reported to the UG office.
- Plagiarism checking will be performed against **all previous batches!**
- Potential penalty (source: UG office)
 - $\geq 10\%$ - **FAIL**
 - $< 10\%$ - max: **reduction in letter grade**

Assessments

Name	Percentage
Coursemology (CA)	30%
Capstone project	10%
Midterm	30%
Final Exam	30%

Gamified CA (30%)

Formative

- The continuous assessment (CA) component is gamified via experience points (EXP).
- 1,000 EXP = 1 Level
- **Your final level is your CA grade**
 - Capped at 30, or 30,000 EXP
 - Only need ~85% of the mandatory work to achieve Level 30.

Gamified CA (30%)

Formative

CA component	Interval	EXP	Bonus EXP
Lecture	Weekly	-	150 EXP bonus!
Training	Weekly	550 EXP	100 EXP early bonus!
Tutorial	Weekly	1000 EXP	
Problem Set	4 PSs, every ~2 weeks	4200 EXP	
Others (PS0, Surveys, Extra Practice)	-	-	More Bonus EXP!

1,000 EXP = 1 Level

Max Level is 30

Gamified CA (30%)

Formative

- Lecture
 - 150 bonus EXP if you participate (i.e., answer questions via PollEV)
- Tutorial
 - You'll be given a participation EXP by your TA
 - **800 EXP**: Baseline, for attending the tutorial
 - **1000 EXP**: Give one complete and correct argument with sound reasoning
 - The following are not considered one complete and correct arguments:
 - Asking questions
 - Completing TA's answers
 - Example: TA: "That's why, the environment is ...", Student (chime in): "stochastic".
 - One-liner answer

Capstone Project (10%)

Exploration

Develop an AI agent to solve a problem.

- You can use any techniques covered in the course, including "classical" AI, "classical" ML, or "modern" ML, or explore approaches beyond the course material.
- More details later!

Duration: 1-2 months

Midterm and Final

Summative

Closed book, open-sheet

Focus on application, not memorization

- **Midterm (30%)**

- Date/Time: Monday, 2nd March 2026, from 18:30pm – 20:30pm
- Venue: MPSH 1A & 1B

- **Final (30%)**

- Date/Time: Saturday, 2nd May 2026 (Exam Week), 13:00pm – 15:00pm
- Venue: TBA

Late Policy (Problem sets)

Penalties:

- Up to 1 hour, 0%
- Up to 24 hours, 20%
- Up to 3 days, 30%
- Beyond 3 days, 50%

If you need an extension (for valid reasons), please ask early

Common Mistakes (Problem sets)

- **Debugging algorithms with Jupyter Notebook**
 - Should use IDE (e.g., Visual Studio Code, IDLE) for PS0 and all other non-ML problem sets
- **Not reading documentations and reinventing the wheel**
 - There are many helpful functions built-in
- **Coding directly on Coursemology**
 - Should code and test locally

Consultation

- **Walk-in:**
 - Office: COM2-02-06
 - Tuesday, 1-2pm
- **Appointment:**
 - Send an email to rizki@nus.edu.sg



Muhammad **Rizki** Maulana
rizki@nus.edu.sg
Deep Reinforcement Learning

Consultation

- **Walk-in:**
 - Office: COM2-04-29
 - Tuesday, 2-3pm.
- **Appointment:**
 - Send an email to cqtfpr@nus.edu.sg



Patrick Rebentrost
cqtfpr@nus.edu.sg
Quantum Computing, Quantum
Machine Learning

Break

**THINGS ARE
GOING TO GET
WEIRD**

TEDEd



Intro to Artificial Intelligence

Outline

- What is AI?
- Intelligent Agents
 - PEAS Framework
 - Properties of Task Environment
 - Structure of Agents
 - Exploration vs Exploitation
- Designing an Agent (if time permits)
 - Problem solving agents
 - Problem formulation

Outline

- **What is AI?**
- Intelligent Agents
 - PEAS Framework
 - Properties of Task Environment
 - Structure of Agents
 - Exploration vs Exploitation
- Designing an Agent (if time permits)
 - Problem solving agents
 - Problem formulation

Examples of AI?



Image credit:
IEEE Spectrum



Image credit:
Tesla



Image credit:
Guardian

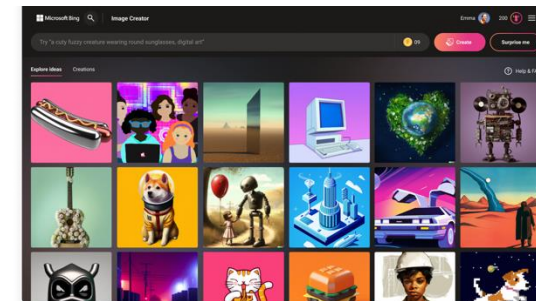


Image credit:
Microsoft



Image credit:
NYTimes



Image credit:
Eden AI

What is AI?

“We define AI as the study of agents that receive percepts from the environment and perform actions.”

- Russel and Norvig, AIAMA 2nd edition

“The science and engineering of making intelligent machines”.

- John McCarthy, 1955

Outline

- What is AI?
- **Intelligent Agents**
 - PEAS Framework
 - Properties of Task Environment
 - Structure of Agents
 - Exploration vs Exploitation
- Designing an Agent (if time permits)
 - Problem solving agents
 - Problem formulation

Intelligent Agents

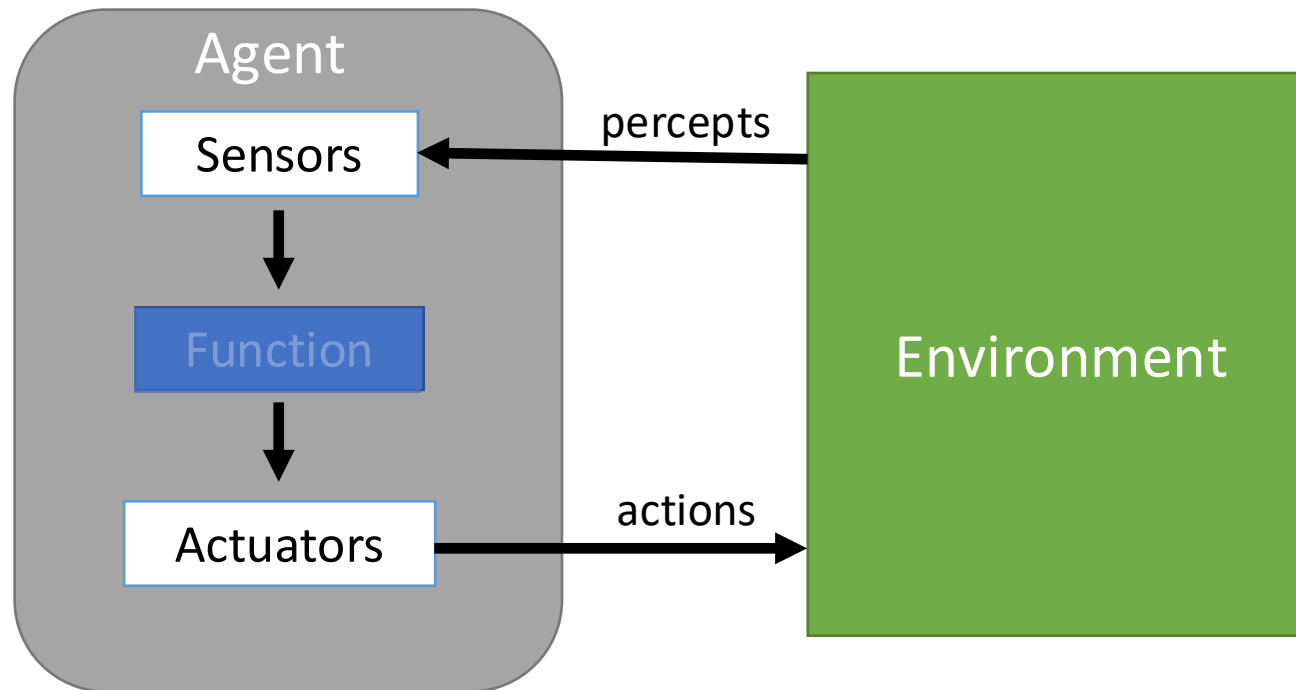
How to design an intelligent agent?



Credit: Axios

Intelligent Agents

Performance Measure, Environment, Actuators, Sensors



Performance Measure

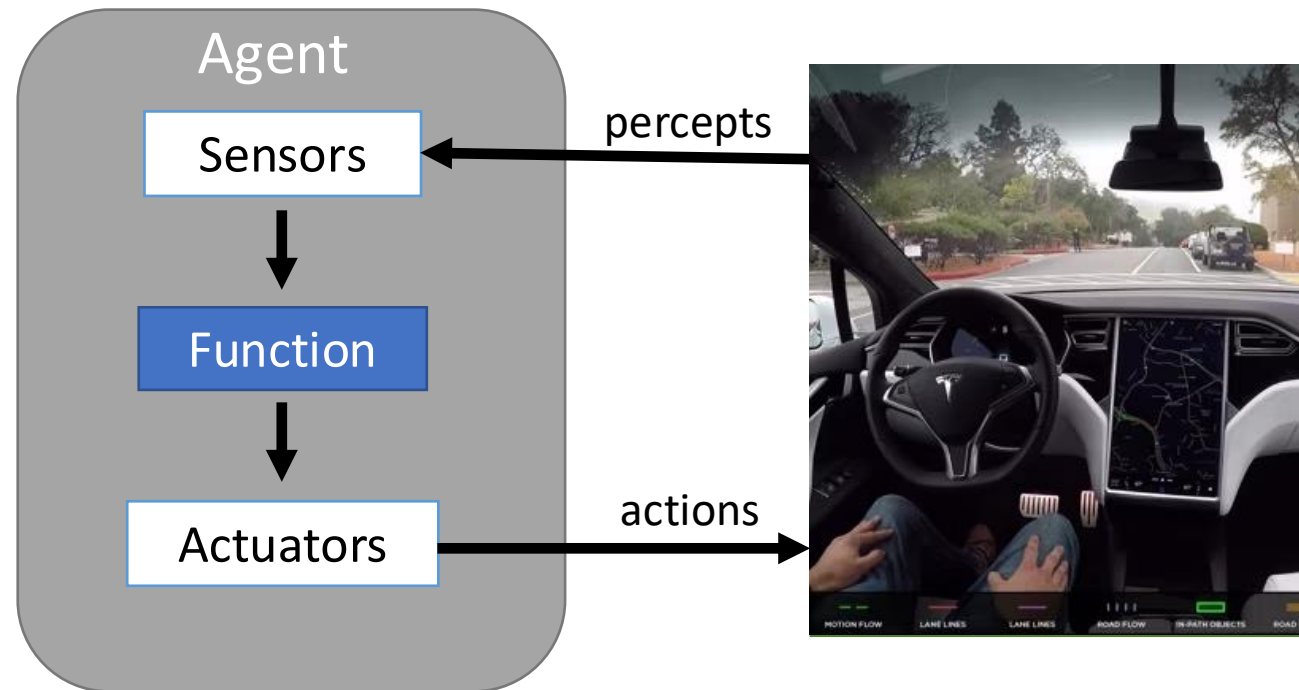
Things to consider:

- Best for whom?
- What are we optimizing?
- What information is available?
- Any unintended effects?
- What are the costs?

A **rational agent** will choose actions that maximize performance measure.

Intelligent Agents: Self-Driving Car

Performance Measure, Environment, Actuators, Sensors



Intelligent Agents: AI Assistant/Chatbot

Performance Measure, Environment, Actuators, Sensors

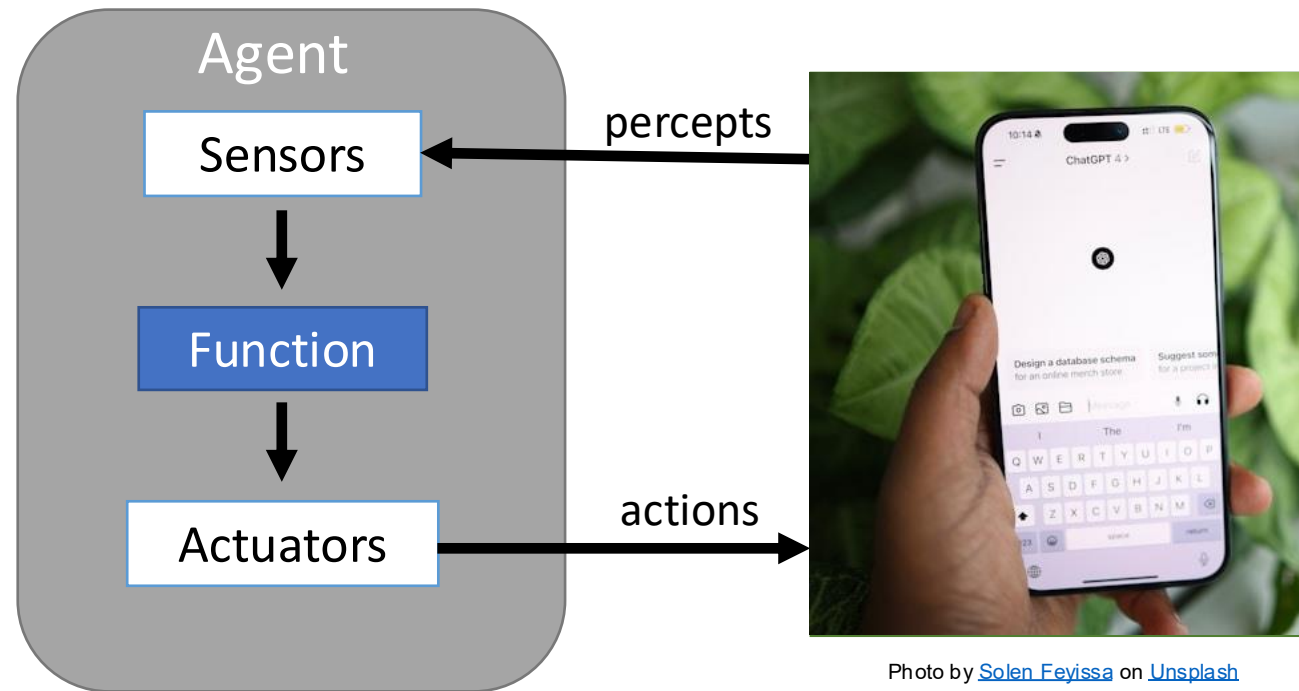
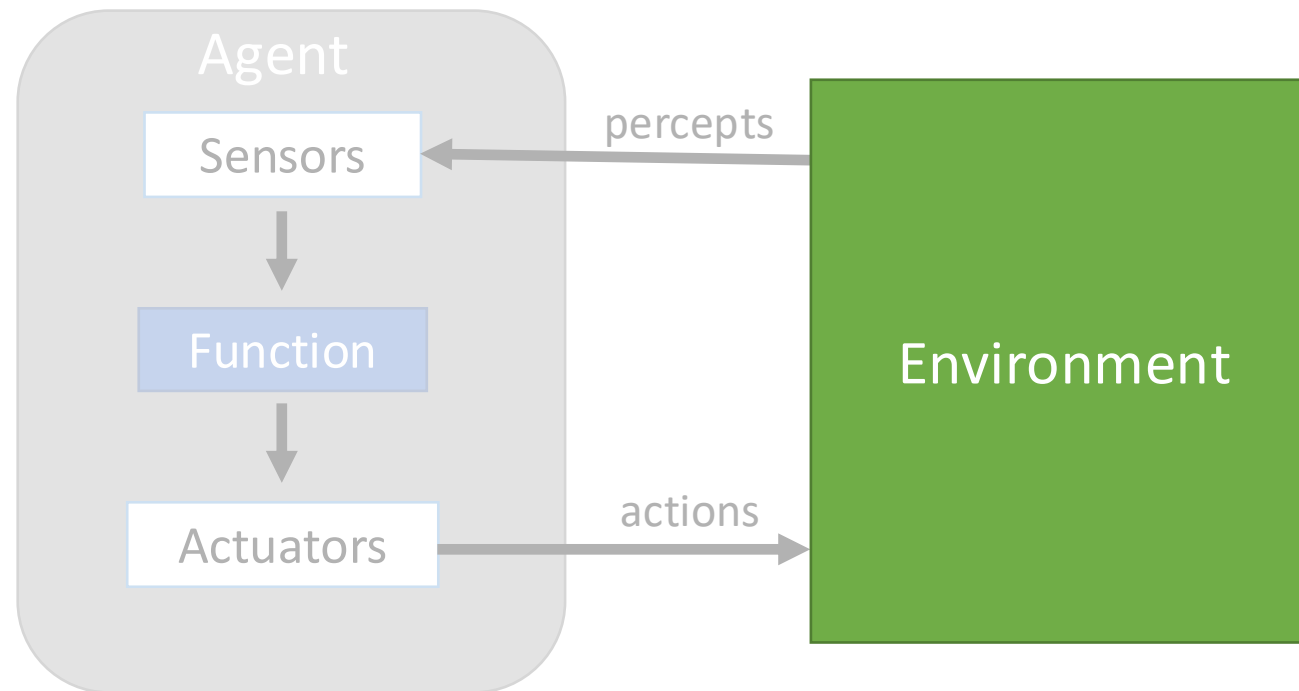


Photo by [Solen Feyissa](#) on [Unsplash](#)

Intelligent Agents



Properties of Task Environment



Full



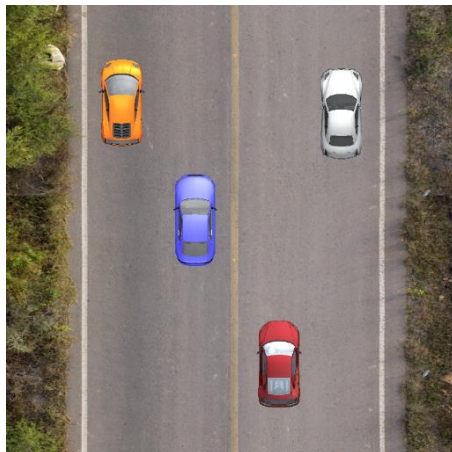
Partial

Fully observable (vs. partially observable)

An agent's sensors give it access to the complete state of the environment at each point in time.

This means the agent can perceive all relevant aspects needed to make decisions; there is no hidden information; No need for internal state to keep track of the world.

Credit: Google Play



Partial

Properties of Task Environment



Multi-agent

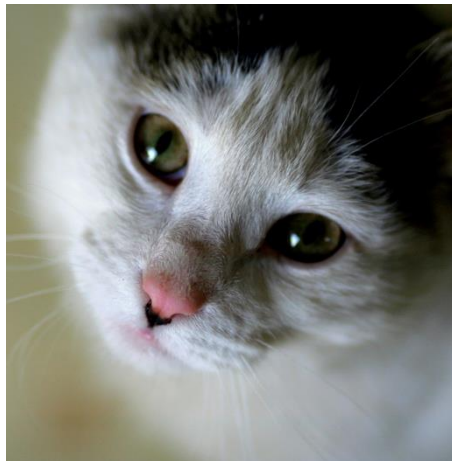


Multi-agent

Single agent (vs. multi-agent)

An agent operating by itself in an environment.

Credit: Wikimedia Commons



Single-agent

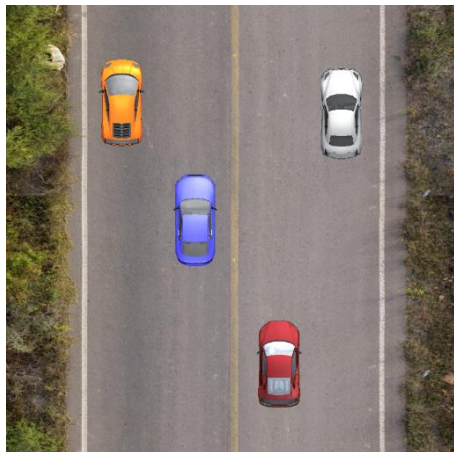
Properties of Task Environment



Deterministic, Strategic



Stochastic, Strategic



Deterministic

Deterministic (vs. stochastic)

The next state of the environment is completely determined by the current state and the action executed by the agent.

If the environment is also dependent on the actions of other agents, then it is also **strategic**. However, if the other agents are predictable, i.e., their strategies are fixed and only dependent on the state, then the other agents can be subsumed as parts of the environment.

Note: In our definition, we ignore uncertainty that arises purely from the actions of other agents in a multiagent environment; thus, a game can be deterministic even though each agent may behave stochastically.

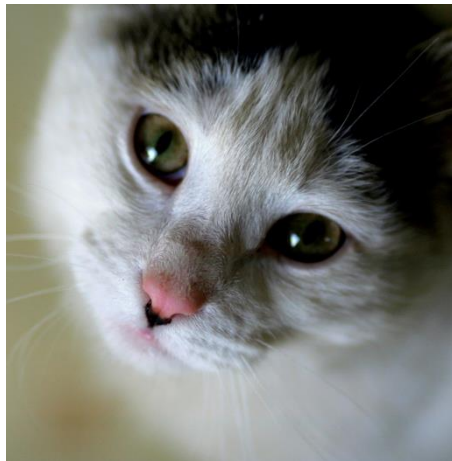
Properties of Task Environment



Sequential



Sequential



Episodic

Episodic (vs. sequential)

The agent's experience is divided into atomic "episodes" (each episode consists of the agent perceiving and then performing a single action), and the choice of action in each episode depends only on the episode itself.

In sequential environments, the current decision could affect all future decisions.

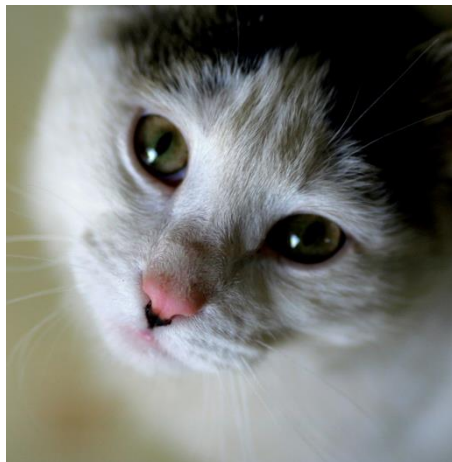
Properties of Task Environment



Static/Semi-Dynamic



Dynamic



Static

Static (vs. dynamic)

The environment is unchanged while an agent is deliberating.

The environment is **semi-dynamic** if the environment itself does not change with the passage of time, but the agent's performance score does.

Properties of Task Environment



Discrete

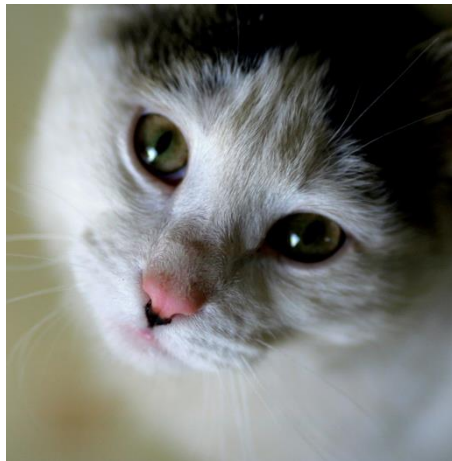


Continuous

Discrete (vs. continuous)

A limited number of distinct, clearly defined percepts and actions.

Credit: Wikimedia Commons



Discrete

Properties of Task Environment

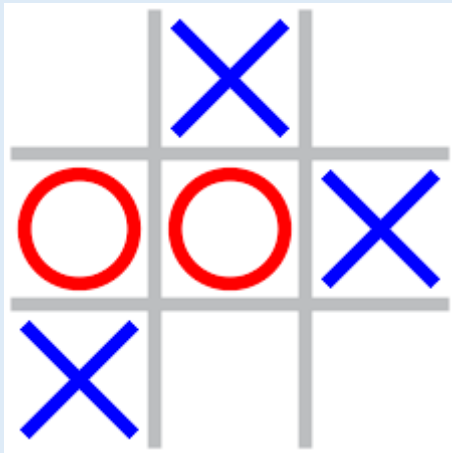
In this course, we will not deal with environments that are:

- Partially-observable
- Stochastic
- Dynamic

Will be taught in more advanced classes, e.g., CS4246

Poll Everywhere

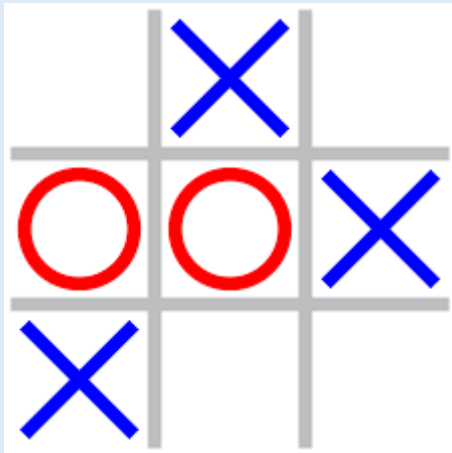
What are the properties of the task environment Tic-Tac-Toe?



Credit: GamesCrafters

Poll Everywhere

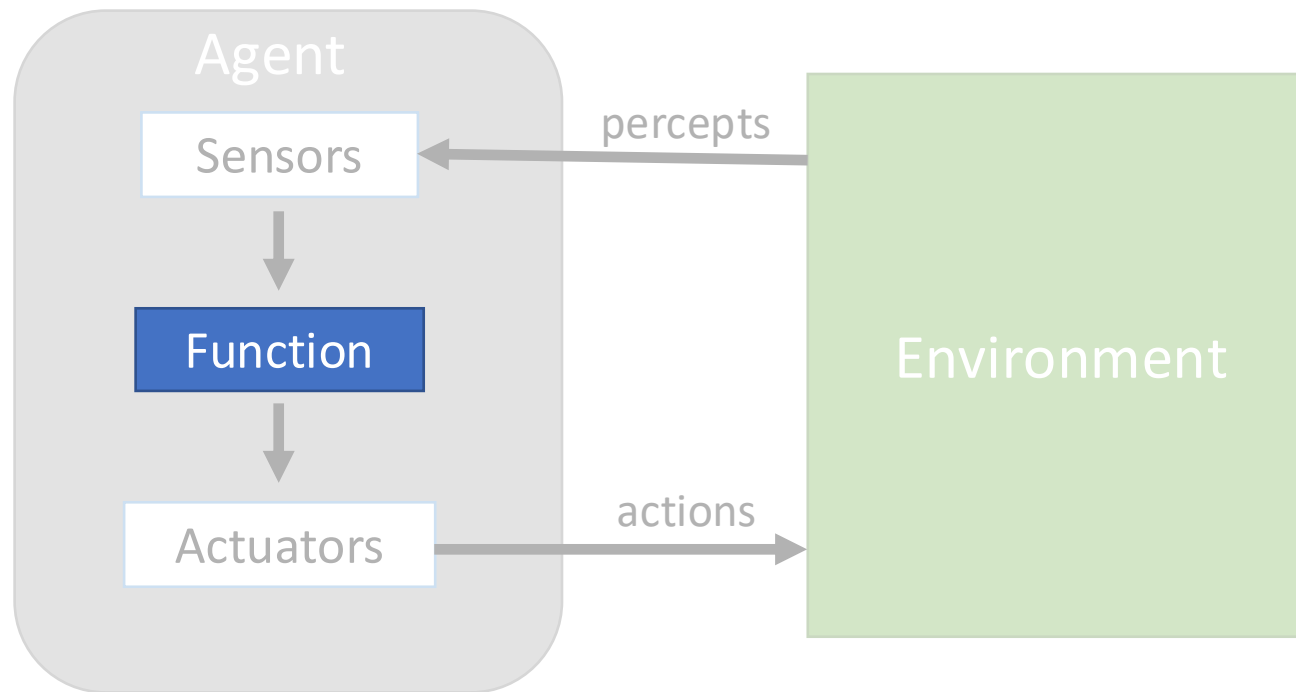
What are the properties of the task environment Tic-Tac-Toe?



Credit: GamesCrafters

- Fully-observable
- Deterministic, Strategic
- Static
- Sequential
- Multi-agent

Intelligent Agents



The **agent function** maps from percept histories $\mathcal{P}$ to actions $\mathcal{A}$:

$$f: \mathcal{P} \rightarrow \mathcal{A}$$

The **agent program** is an implementation of the agent function.

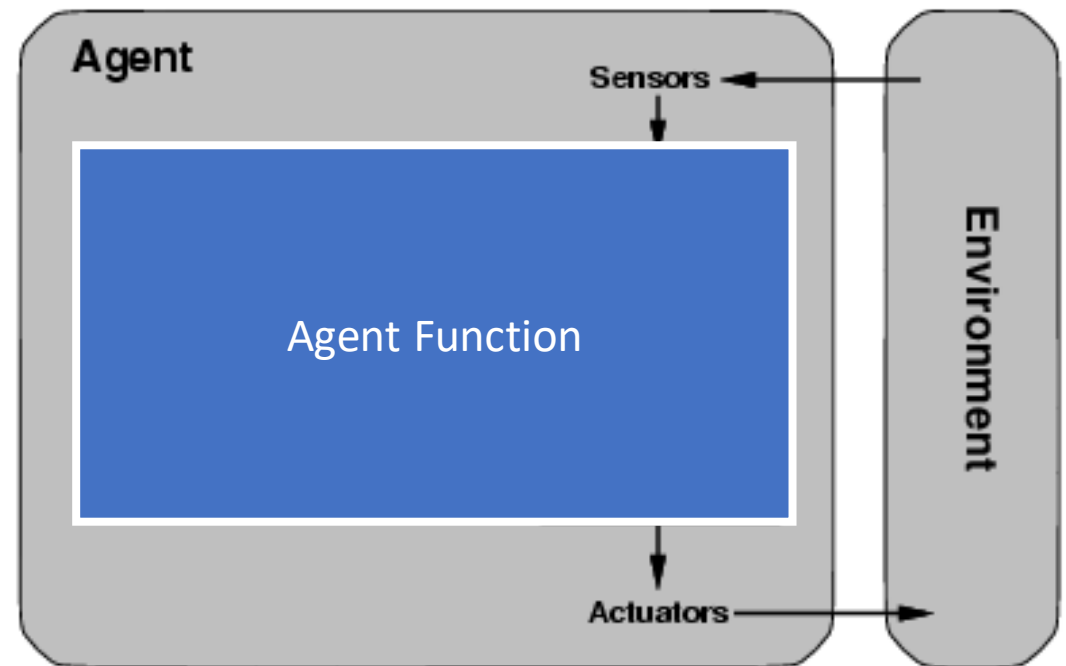
For a **percept**, the program runs on physical architecture to produce an **action**.

The Structure of Agents

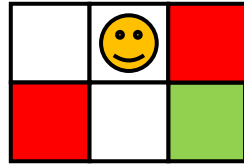
Given PEAS, an agent is completely specified by the **agent function**.

Common agent structures

- Simple reflex agents
- Goal-based agents
- Utility-based agents
- Learning agents



Simple Reflex Agent



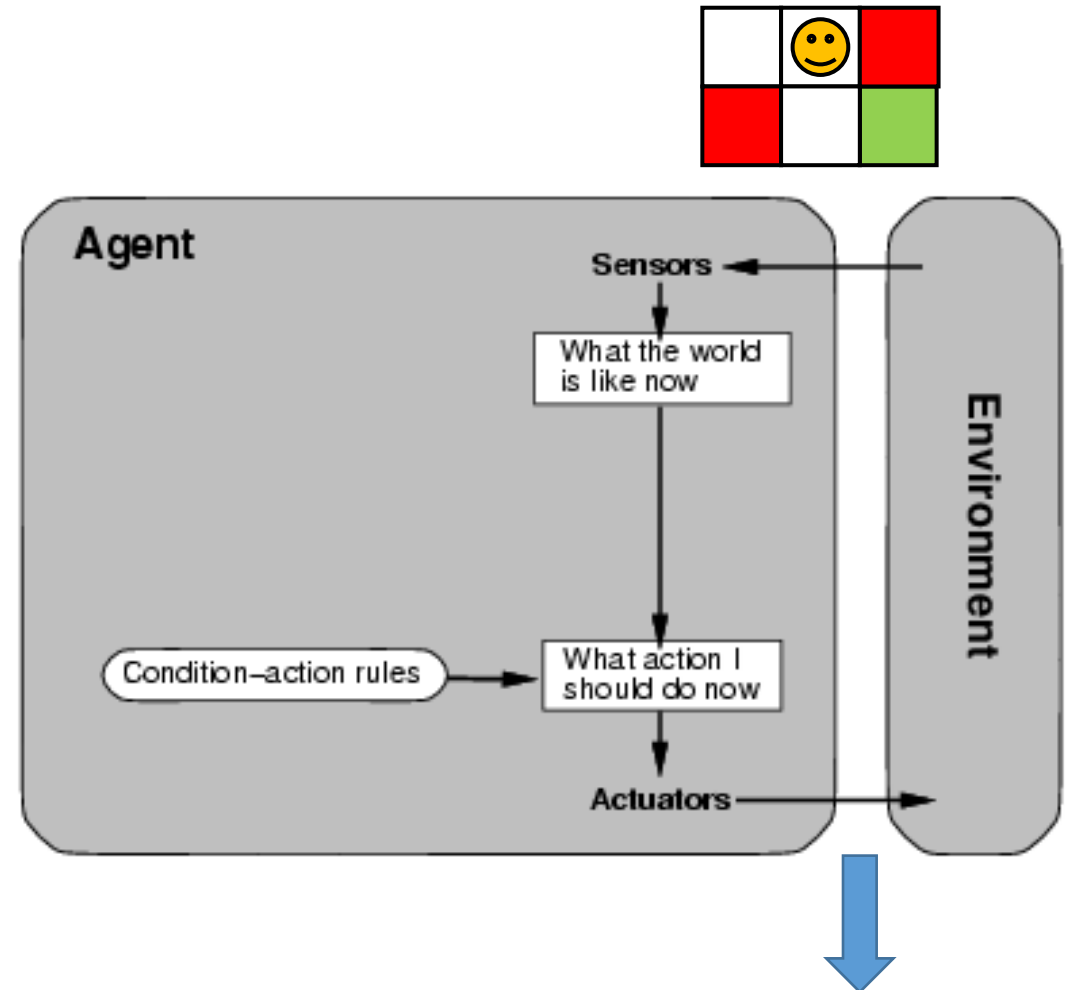
Condition-action rule

If up empty: up

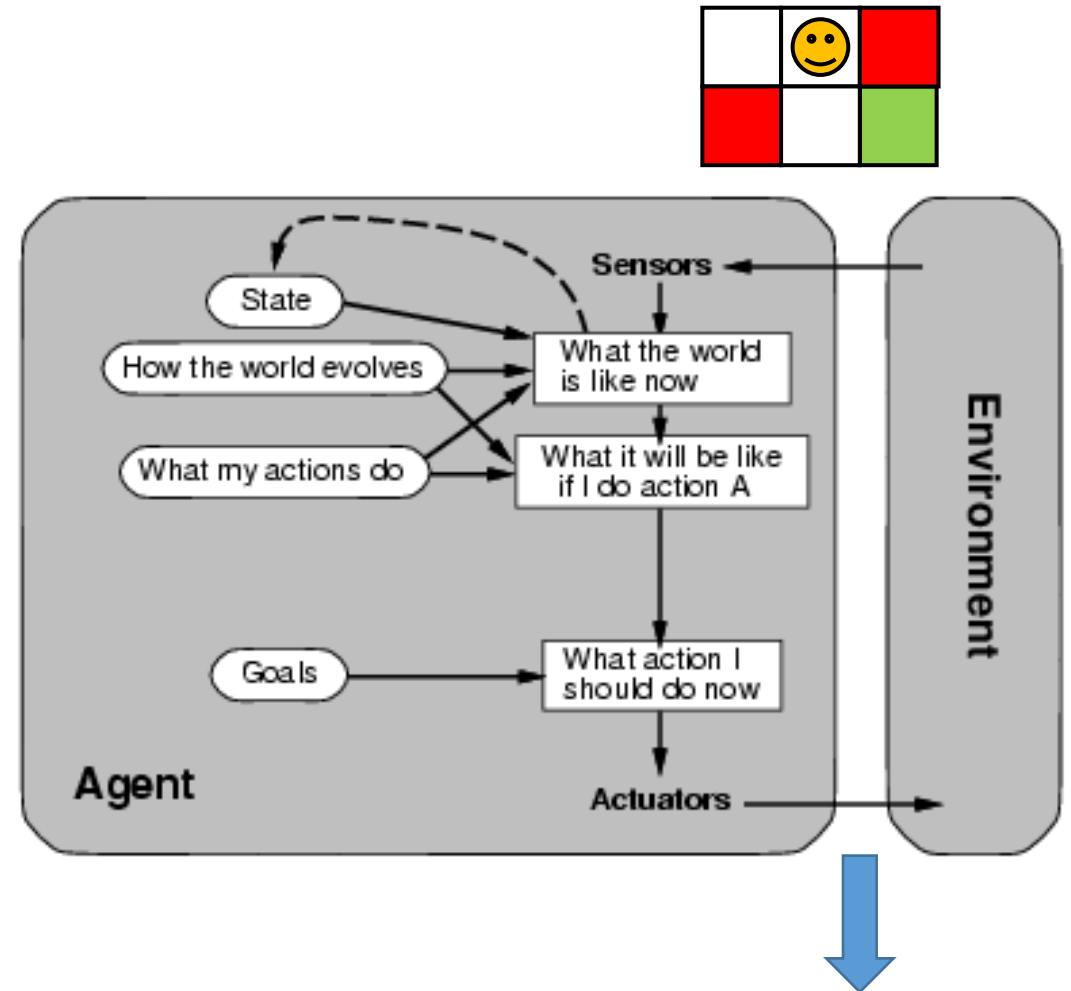
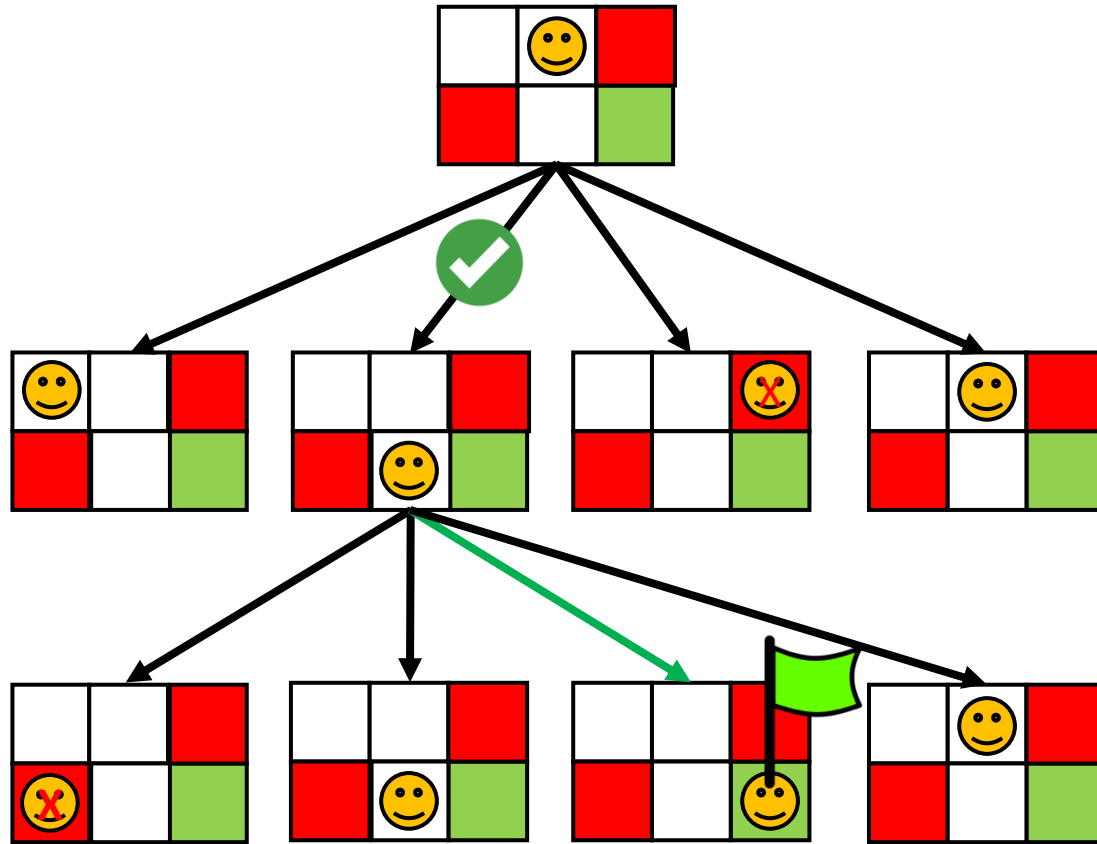
If right empty: right

If down empty: down

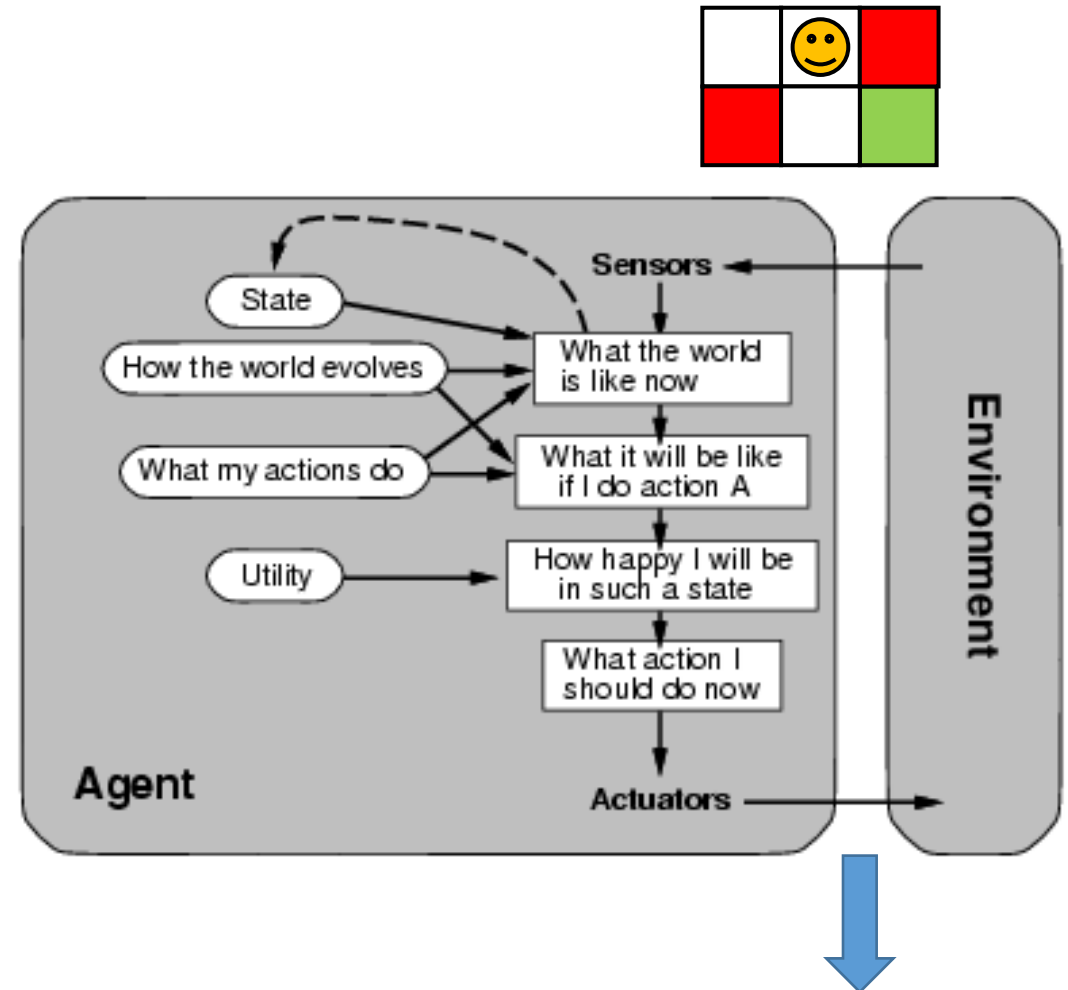
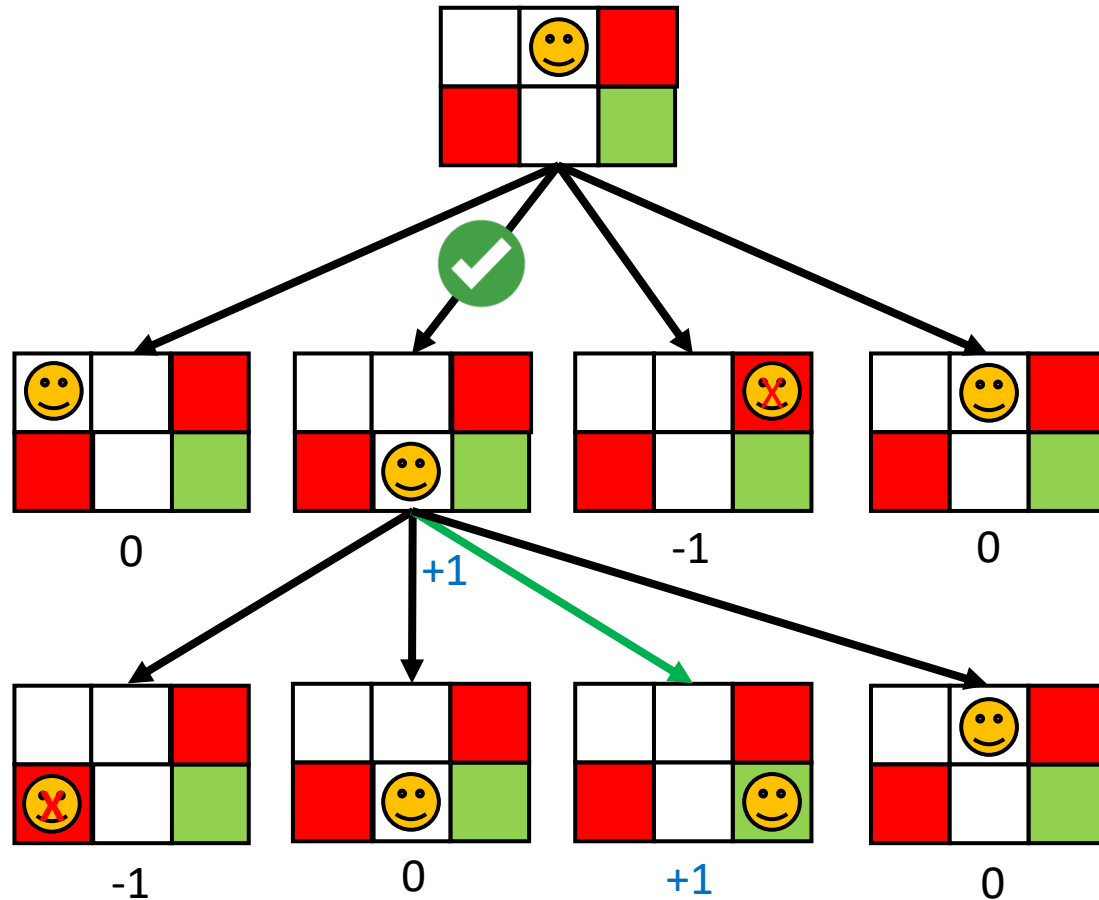
If left empty: left



Goal-based Agent



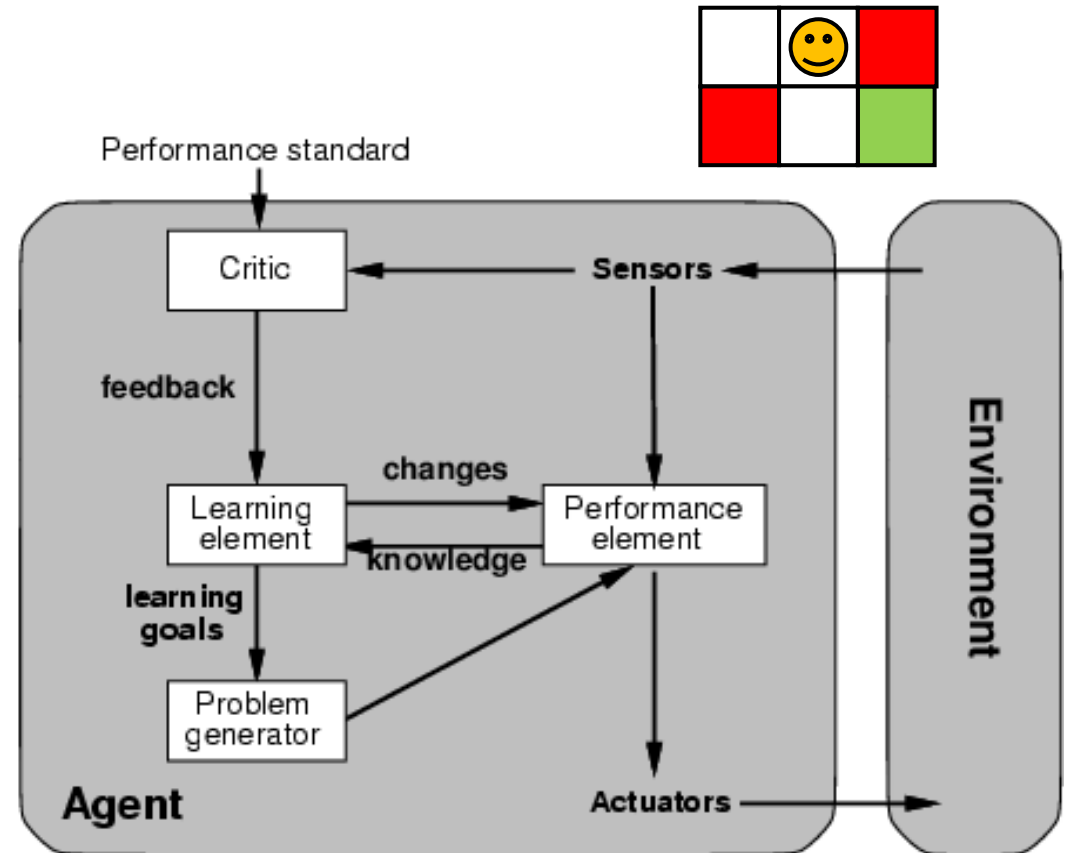
Utility-based Agent



Learning Agent

A learning agent is an agent designed to improve its performance over time by using experience. It does so by adjusting how it selects actions so that it performs better with respect to a given performance measure in its environment.

- **Performance element:** Chooses actions based on the agent's current knowledge.
- **Learning element:** Improves the performance element using feedback and experience.
- **Critic:** Evaluates how well the agent is doing relative to a performance standard and supplies feedback to the learning element.
- **Problem generator:** Proposes exploratory actions that help the agent discover useful new information that can lead to better performance.



Other Agent Structures

- Model-based reflex agent
- Combinations of common agent structures (e.g., Alpha Go)

Intelligent Agents: Exploration vs Exploitation

Learn more about the world

Maximize gain based on
current knowledge



Credit: Axios

Intelligent Agents: Exploration vs Exploitation

Learn more about the world

Maximize gain based on
current knowledge



Credit: Amazon

VS



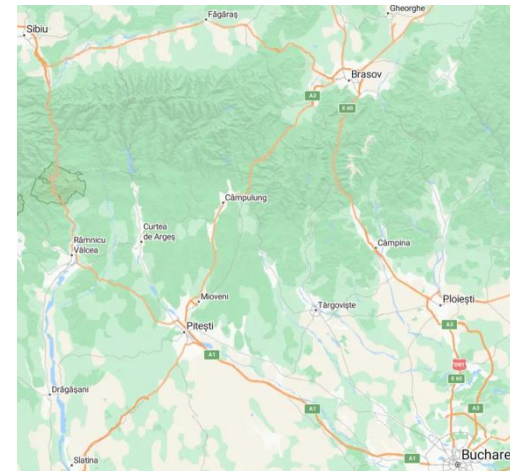
Credit: Indomie

Outline

- What is AI?
- Intelligent Agents
 - PEAS Framework
 - Properties of Task Environment
 - Structure of Agents
 - Exploration vs Exploitation
- **Designing an Agent (if time permits)**
 - Problem solving agents
 - Problem formulation

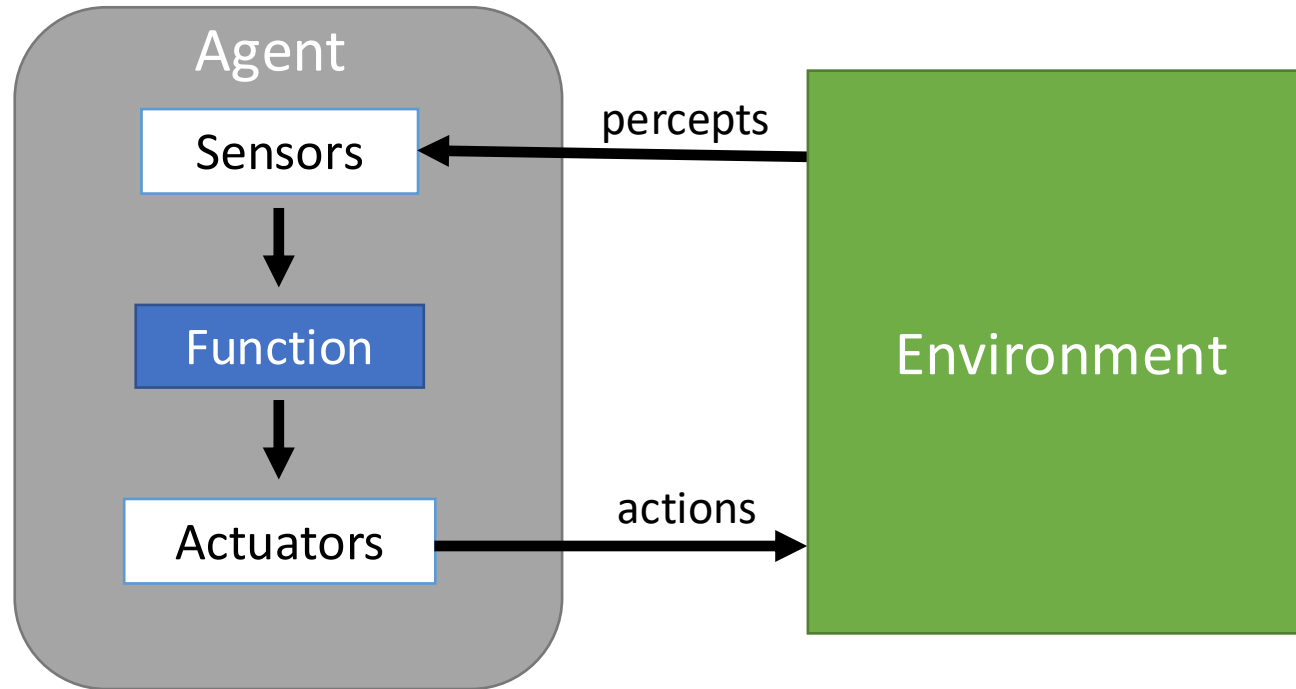
Preliminary: Search Problems

- A **search problem** refers to a type of problem where the goal is to find a state (or a path to a state) from a set of possible states by exploring various possibilities.
- Examples:
 - Path finding
 - Puzzle solving



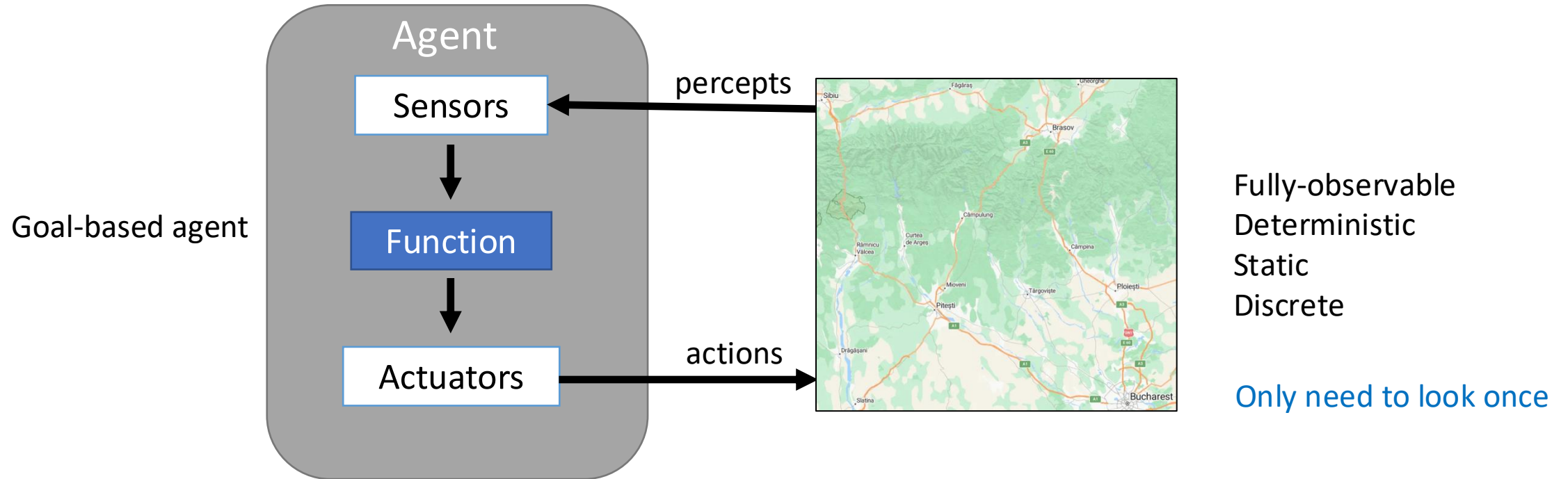
9	1	3				5		
6		7					2	4
	5			8			7	
	7	9						
		2		9			4	3
					4		9	
	4				1	9		
7		6			9			5
		1			6	4		7

Designing an Agent



Designing an Agent

for problems that can be solved by **searching**. E.g., path finding



How is search used by the agent?

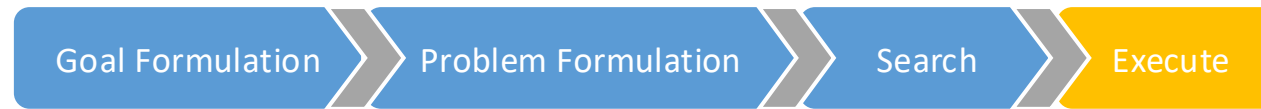
Designing an Agent

To solve a problem using search, the agent needs to have:

- A goal, or a set of goals
- A model of the environment
- A search algorithm

Problem Solving Steps

What does the agent want? How does the world work? How to achieve it?



Problem Formulation in (Systematic) Search

- **States:** Each state corresponds to an actual/specific situation or configuration of the problem domain.
 - **Representation invariant:** ensure that the *abstract states* have corresponding *concrete states*.
- **Initial State:** The starting configuration
- **Goal Test / State(s):**
 - A function that checks whether a state is the goal state, or
 - A collection of goal state(s)
- **Actions**
 - For every state, we have a set of actions, denoted by $actions(state)$ with size $|actions(state)|$.
 - An integer b is called a branching factor, if for all states $|actions(state)| \leq b$.
- **Transition model**
 - Given a state and action, the transition function produces the next state
 - $new_state = transition(state, action)$
- **Action cost function:** map state and action to a cost

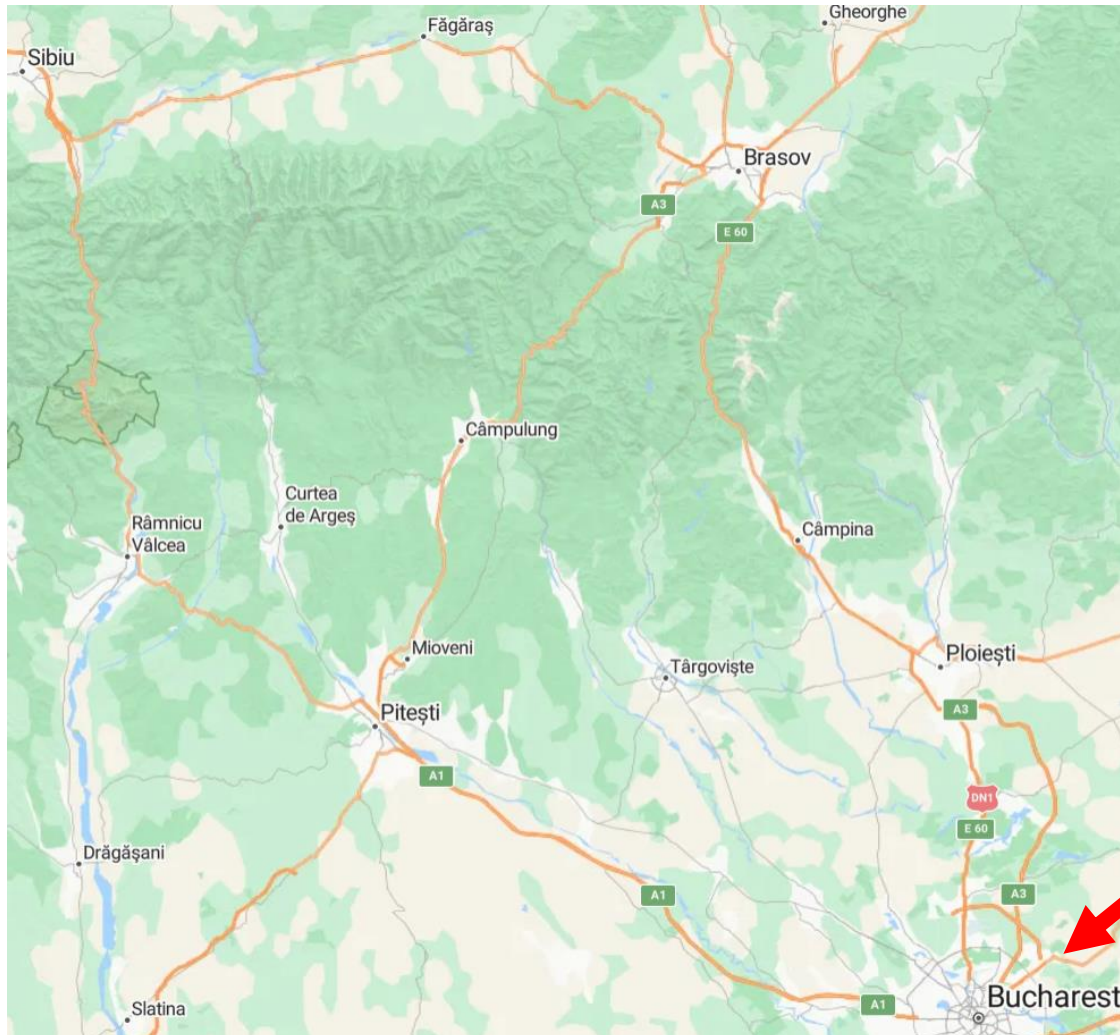
Example: Romania

Goal Formulation

Problem Formulation

Search

Execute



Go to Bucharest



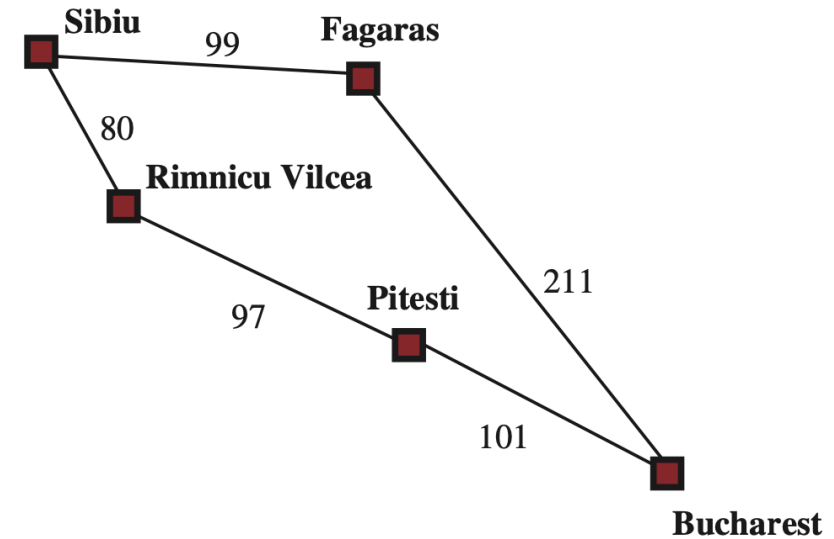
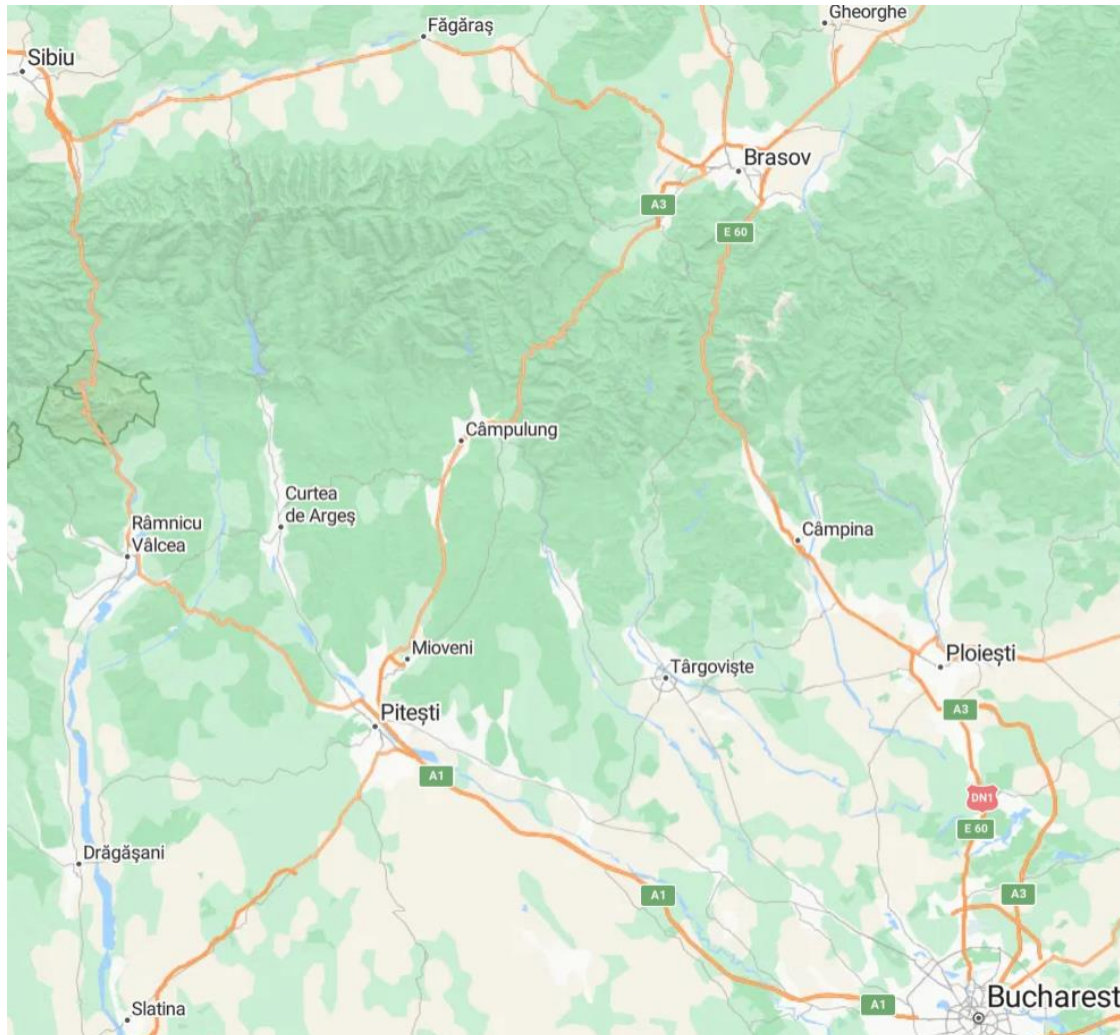
Example: Romania

Goal Formulation

Problem Formulation

Search

Execute



Problem Formulation

- States
- Initial state
- Goal state(s)/test
- Actions
- Transition model
- Action cost function

Cities: {at Sibiu, at Pitesti, ...}

at Sibiu

at Bucharest

Go to a neighboring city

Current state = selected city

See edges

Example: Sudoku

Goal Formulation

Problem Formulation

Search

Execute

9	1	3				5		
6		7					2	4
	5			8			7	
	7	9						
		2		9			4	3
					4		9	
	4				1	9		
7		6			9			5
		1			6	4		7

9x9 Integer Matrix

Problem Formulation

- States
- Initial state
- Goal state(s)/test
- Actions
- Transition model
- Action cost function

All possible assignments of numbers to a 9x9 matrix

Partially filled matrix

Check if all constraints are satisfied

Fill matrix

Current state = filled matrix

1

Summary

- AI: constructing agents to solve complex tasks
- **PEAS** Framework:
 - **P**erformance measure: define “goodness” of a solution
 - **E**nvironment: define what the agent can and cannot do
 - **A**ctuators: outputs
 - **S**ensors: inputs
- Agent function is sufficient to define an AI agent
- Common agent structures:
 - Reflex, goal-based, utility-based, learning
 - There are others! Can mix-and-match!
- **Exploration** vs **exploitation**
- Problem solving agents

Coming Up Next Week

- **Formulating search problems**
- **Uninformed search algorithms**
 - Breadth-first search
 - Depth-first search
 - Uniform-cost search
 - ...
- **Informed search algorithms**
 - A* Search
 - Heuristics
 - ...

To Do

- **Lecture Training 1**
 - +550 EXP
 - +100 Early bird bonus
- **Problem Set 0 (completely optional!)**
 - +900 bonus EXP, until 7 February (end of week 4)

Appendix

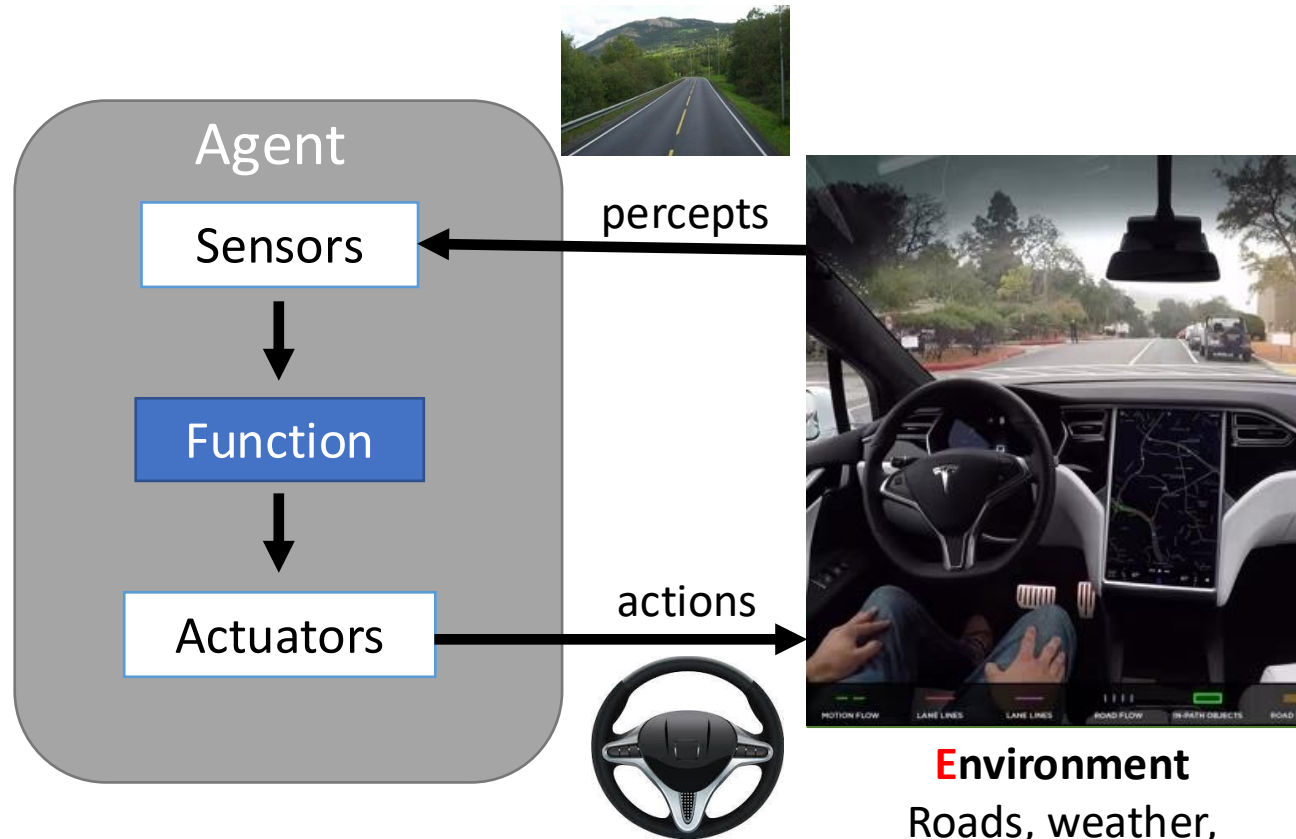
Intelligent Agents: Autonomous Driving

Sensors

- Camera
- LIDAR
- Speedometer
- ...

Actuators

- Steering wheel
- Accelerator
- Brake
- ...



PEAS

Performance Measure, Environment, Actuators, Sensors

Performance Measure

- Safety
- Speed
- Legal
- Comfort
- ...

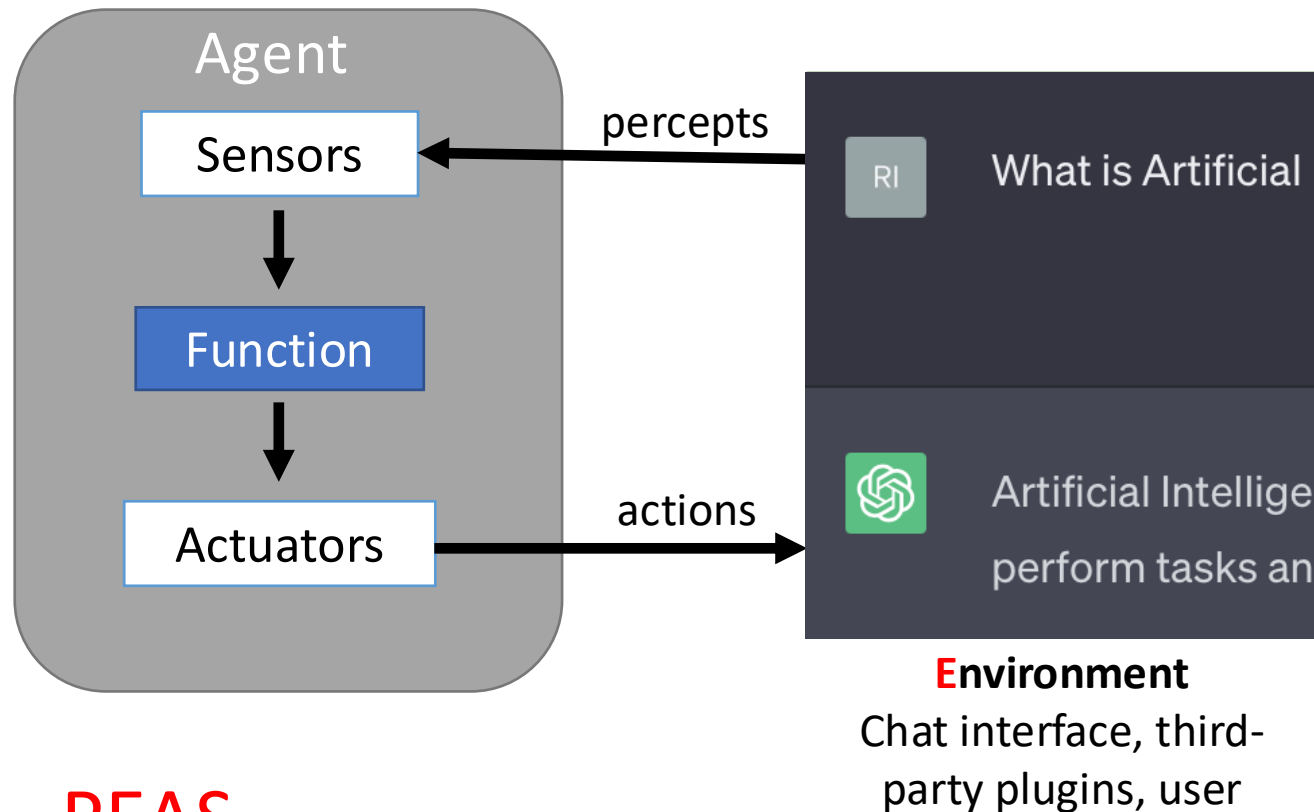
Intelligent Agents: AI Assistant/Chatbot

Sensors

- Text input
- Chat history
- Context
- ...

Actuators

- Text output
- Image output
- API
- ...



PEAS

Performance Measure, Environment, Actuators, Sensors

Performance Measure

- Correctness
- Conciseness
- Legal
- Safety
- ...